Library build with task selection: first comparison and noise floor

Date: 2026-10-14 onwards (section 7 is updated as results arrive). One RTX 5090. Model: `claude-sonnet-5` for every writer and repair agent. Figures: `figs/` (make them with `make_figures.py`). Code: `scripts/taskselect/library_loop.py`.

Summary

- We built a skill library from nothing in two ways, one run each: the next task to repair chosen by **disagreement** (`lex`), and chosen **at random among unsolved tasks** (`unsolved`).
- **No difference can be claimed.** Each round runs 27 programs (9 tasks x 3) for 15 trials each, 405 trials. The final pass rate is 0.141 (`lex`) against 0.052 (`unsolved`), but three independent generations of the *same* empty library already range from 0.035 to 0.091, and every 95% interval of the gap contains 0.
- Two clear library effects were seen, one in each arm, and both are real mechanisms, not selection effects: a perception rule that was used and helped, and a perception prompt that helped and then was not reused.
- A pipeline defect (point-prompt API misuse, never fixed in the writers' instructions) crashed up to 30% of the `lex` trials in rounds 1 to 3. It biases the comparison against `lex`. It is fixed in all later runs.
- **Campaign in progress (section 7):** repetitions of both arms with 30 probe seeds and 5 repairs; 2 of 5 are finished. With the first 3 programs per task `lex` ended higher in both (+0.032, +0.059). Re-measuring each final library with 3 more programs per task (6 in total) **removed the gap** (-0.009, +0.002): the lead was writer luck. So far there is no evidence that disagreement selection beats random selection. All four runs end above their round 0.

Proposition and hypotheses

Setting. A language model writes a Python program for a robot task from a skill library (markdown files of rules and helpers). When programs fail, an agent repairs one task, and the lesson is written back into the library. The library is the only thing that carries knowledge from one round to the next. Repairing is expensive (about an hour of agent and simulator time per task), so a build repairs only a few tasks. **Which task to repair next decides what the library learns.**

Proposition. The most informative task to repair is the one where the current programs **disagree**: some pass and some fail on the same seeds, many unsolved tasks fail at the same stage, and the failures have varied traces. Disagreement says that the task is neither hopeless nor already solved, and that a fix at a shared failure stage will carry over to other tasks. Repairing by this rule (`lex`) should build a better library than repairing a random unsolved task (`unsolved`) at the same number of repairs.

Hypotheses (all about the library after the same number of repairs, same tasks, same writers):

id	statement	measure	how it can fail
H1 (primary)	the final library of `lex` gives a higher pass rate than that of `unsolved`	mean pass rate over 27 programs (9 tasks x 3), and over 6 programs per task, on 30 seeds	gap within the noise, or negative
H2	`lex` improves a library faster along the rounds	gain over round 0, mean over rounds 1 to 5	no gain difference
H3	`lex` lessons transfer: tasks that were not repaired improve	per-task passes, unrepaired tasks	gains only on repaired tasks
H0 (noise)	one run's mean moves by 0.03 (SD) with no change in the library, so an effect smaller than the detectable gap cannot be shown	six round-0 generations from the empty library	(this is a measured fact, not a claim)

What can be concluded. The smallest gap that two arms can reliably show (5% level, 80% power) is 0.106 with 1 repetition, 0.075 with 2, 0.061 with 3, 0.053 with 4, 0.043 with 6. A result below that is reported as "not shown", not as "no effect". The harness idea (section 8) is a second, separate proposition and is not tested yet.

Method: the library, the loop and the update rule

The library. Markdown files in `.claude/libero/skills/` (`grasp`, `localize`, `manipulation`, `transport`, `README`). Writers read them as context; programs never import them. A build starts from the empty library `L00`, and each repair produces the next one (`L01`, `L02`, ...). Every version is stored in `outputs/<run>/library/Lnn`.

One round (pseudo-code).

```
L <- empty library                                  # L00
```

```
for t in 0 .. R-1:                                  # R = 5 repairs in the campaign
    for task in pool:                               # 9 swap tasks
        programs[task] <- K writers (K=3), each a fresh zero-shot program from L, differing in strategy
    for program in programs: outcome[program][seed] <- run in simulator, seeds 51..80    # 27 x 30 trials
    unsolved <- tasks with no program that passes all seeds
    task <- select(rule, unsolved, outcome, traces)          # lex or unsolved (below)
    best, findings <- repair(task, start = best program of task, dev seeds 51..65, <= 5 iterations)
    L <- promote(L, findings)                       # the library update, below
    save L as L(t+1)
final probe: write K programs per task from the last library, run them on seeds 51..80
```

Task selection.

```
select(unsolved, outcome, traces):
    if rule == "unsolved": return uniform_random(unsolved)                    # baseline (salted seed)
    for task in unsolved:
        split(task)  = mean over seeds s of entropy of (number of programs that pass seed s)
        shares(task) = number of unsolved tasks whose most common failing stage equals task's
        trace(task)  = entropy of the (failure stage, printed failure tag) signatures of all trials of task
    return argmax over unsolved of the tuple (split, shares, trace)           # lexicographic, ties broken by salted ra
```

Failure stage of a trial is read from its API trace only: `crash`, `perceive` (object not found), `no_motion`, `no_close`, `air` (closed on nothing), `slip` (held, then lost), `held_end`, `released` (placed wrong), `pass`. `LEX.md` has a worked example.

Repair (the fix loop).

```
repair(task, start, dev seeds):
    read pass and fail traces of the 3 probe programs; compare programs that pass a seed another fails
    for i in 1 .. 5:
        write one task-level program (no seed-specific branch, no hard-coded coordinates) from the best evidence
        run it on the 15 dev seeds; diagnose its failed seeds from traces
    kept <- iteration with most passes (ties: the later one)
    write findings.md: root causes, pass count of every iteration, "Generalizable Patterns"
```

Library update rule (promotion). After the repair, the coordinator reads `findings.md`, edits the skill files and logs it in a ledger (`skill_promotions.jsonl`: files changed, reason). Rules in force:

1. A lesson is added as a short rule with its trigger and evidence, in the file of its category (grasp, localise, place, carry). Generic patterns are preferred over task facts.
2. No rule is ever removed or corrected: the library only grows (4.7 KB after the first repair, 17.6 KB after five in one run).
3. The kept program's pass count on the dev seeds is the only evidence required. There is no test on other seeds, which is the weakness section 8 addresses.
4. Skills may use the allowed APIs only (no simulator state, no `.bddl`/`.xml` reads).

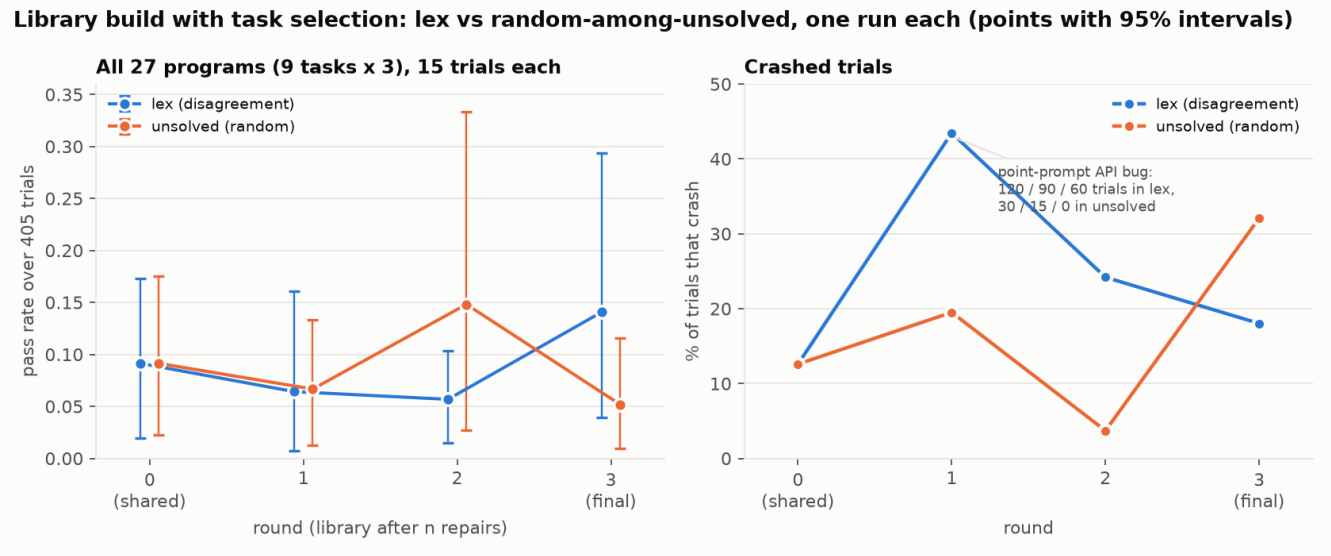
Protocol and controls.

item	value
pool	9 swap tasks: goal (3), spatial (3), object (3); 21 more swap tasks are held out and unused so far
probe seeds	51-80 (30), every round; repair development seeds are 51-65
repetition	round 0 is written once and shared by both arms; arm order alternates; 5 repetitions planned
same in both arms	model (`claude-sonnet-5`), K, writers' prompts, repair budget, seeds, API facts note
differs	only the selection rule
noise control	six independent round-0 generations from the empty library, and a re-measurement of each final library with 3 more programs per task

Setup

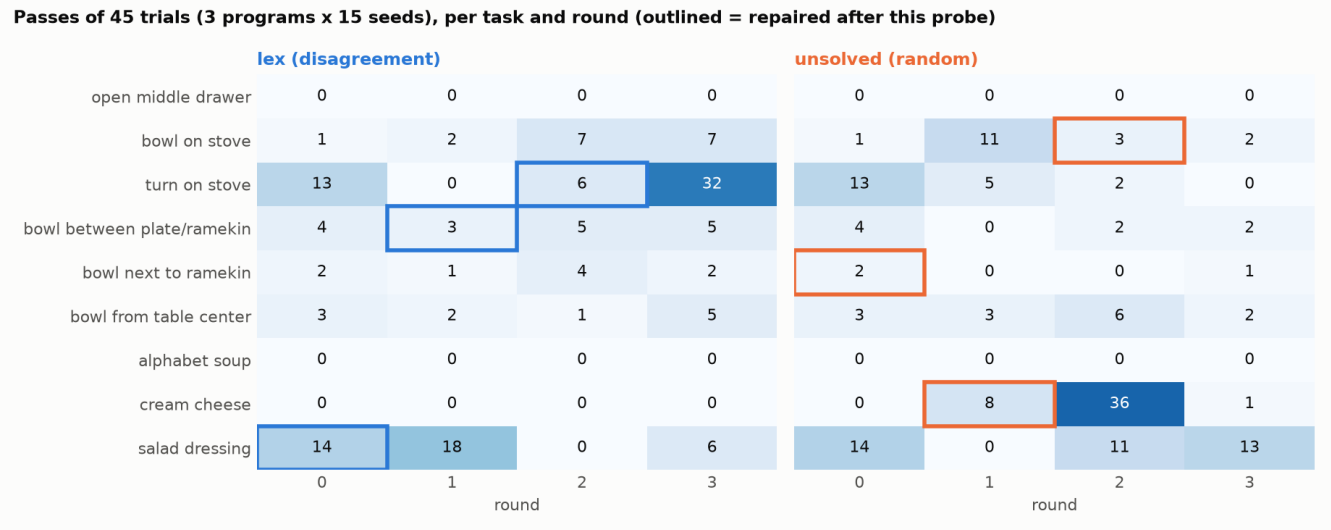
item	value
pool	9 swap tasks: goal (3), spatial (3), object (3), `library_9_swap`
start	empty skill library `L00`; round 0 shared by both arms (27 programs, 9 patched by hand for API errors, see Defects)
per round	3 fresh zero-shot programs per task from the current library, probed on seeds 51-65 (405 trials)
repair	one task per round, up to 5 iterations from its best program, then skill promotion
rounds	3 repairs, then a final probe (4 probes per arm)
selection	`lex`: highest per-seed split, then most tasks sharing the failure stage, then trace variety (explained, with a worked example, in `LEX.md`). `unsolved`: uniform among tasks with no program passing 15 of 15
differences between arms	`unsolved` probed with 4 workers, `lex` with 2 (speed only; outcomes are not deterministic at either)

1. The two arms



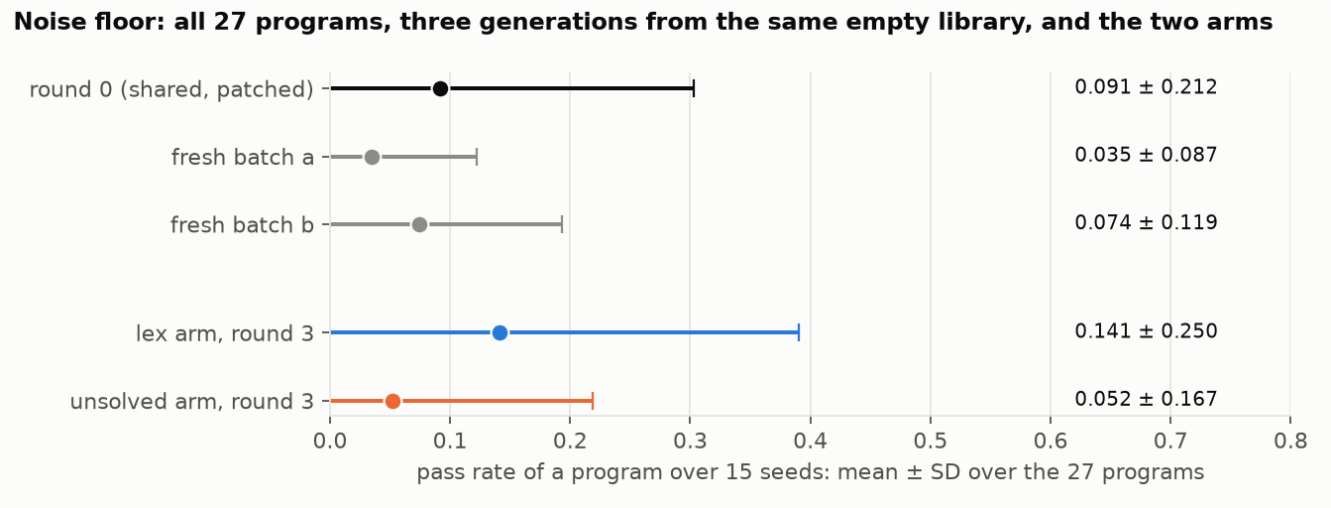
round	`lex` pass rate	`unsolved` pass rate	`lex` crashed trials	`unsolved` crashed trials
0 (shared)	0.091	0.091	13%	13%
1	0.064	0.067	43%	19%
2	0.057	0.148	24%	4%
3 (final)	0.141	0.052	18%	32%

- "Pass rate" is the passes of all 27 programs (9 tasks x 3 programs, 15 trials each) divided by the 405 trials.
- The best program per task is not used as a metric: a maximum over 3 noisy programs moves by 0.07 to 0.09 between identical generations.
- Solved tasks (one program passes 15 of 15): at most 1 of 9 in any round, in both arms.
- Repairs: `lex` picked salad dressing, bowl between plate and ramekin, turn on the stove. `unsolved` picked bowl next to the ramekin, cream cheese, bowl on the stove. Each arm solved one of its three repair tasks.
- The sign of the difference flips between rounds (+0.089 in round 3, -0.091 in round 2).



The per-task view (passes of 45 trials: 3 programs x 15 seeds) shows how unstable single tasks are: salad dressing in `lex` goes 14, 18, **0**, 6; cream cheese in `unsolved` goes 0, 8, **36**, 1. Neither can be explained by the library alone.

2. Noise floor



Three independent generations of programs from the same empty library, same pool, same seeds. Each generation is 27 programs (9 tasks x 3) with 15 trials each, 405 trials. The plot shows, for each generation and for the final round of each arm, the mean and SD over the 27 programs of each program's pass rate (passes of 15).

generation	mean ± SD over the 27 programs	passes of 405	uncaught exceptions
round 0 (shared, 9 programs patched by hand)	0.091 ± 0.212	37	51
fresh batch a	0.035 ± 0.087	14	63
fresh batch b	0.074 ± 0.119	30	0
`lex`, round 3	0.141 ± 0.250		
`unsolved`, round 3	0.052 ± 0.167		

- The SD is large against the mean: most programs pass 0 of 15 and a few pass most seeds.
- The mean of the three generations is 0.067 (SD between generations 0.029; the pooled per-task estimate is 0.030). That is the noise on one run's mean.
- The clean pair is a against b (same process, no patch): 0.035 against 0.074 (14 against 30 passes of 405).
- The round-0 generation is the highest, and it is the only one that was patched. So part of its gap to a and b is the patch.
- Execution noise adds to this: re-running identical programs on the same seeds changed 11 of 15 per-seed outcomes for one program (earlier control).

Reading the arms against this. The difference between two single runs has an SD of about 0.04.

round	gap `lex` - `unsolved`	95% interval (same tasks, seeds resampled)
1	-0.002	[-0.101, 0.121]
2	-0.091	[-0.291, 0.052]
3 (final)	+0.089	[-0.032, 0.269]

All three intervals contain 0. The final-round gap is about 2 noise SDs, but it appears in **both directions** (round 2 favours `unsolved` by the same amount). With one run per arm, three noise points and several metrics and rounds looked at, that cannot be read as an effect of the selection rule. It does show that library changes or generation variance move the metric by that much between rounds. A comparison needs several repetitions per arm, which is what the campaign now running provides.

3. The two libraries

	`lex` **(L03)**	`unsolved` **(L03)**
files	grasp, localize, manipulation, transport, README: 18 KB	localize, manipulation: 15 KB
edits	additions only, no rule removed or corrected	additions only
general rules	grasp plausibility filter; proprioceptive grasp check; min-thickness filter; landmark exclusion	out-of-vocabulary prompts are not "absent"; wrist-camera re-localisation with a displacement gate; floor-based drop height; learned planner then forced top-down
task-specific	push-flick for the stove knob; `DROP_CLEARANCE = 0.11`	gate values 0.08 and 0.12 m with the cream-cheese basket as evidence

Neither library is clearly better. `lex` covers more skill categories because its three repairs hit different failure kinds.

4. Which library elements measurably helped

element	evidence	verdict
min-thickness filter (`lex`, from the stove repair)	no round-1 or round-2 stove program implements a thickness filter (one comment mentions "flat"); all three round-3 programs do; passes 0/1/5 become 8/10/14	best evidence in the data
describe by appearance (`unsolved`, from the ramekin repair)	all 3 cream-cheese programs in rounds 1 and 2 use the prompt "white and blue box"; round 2 passes 11/15/10	helped, then lost: no round-3 program used it (1 pass of 45), with the rule still in the library
learned planner then forced top-down (`unsolved`, round 2)	programs calling `plan_grasp` rose from 13 to 19 of 27; "No grasp candidates found" 31 times, "All grasp attempts failed" 31 times; crashed trials 4% to 32%	correlation only; harmful in that round
drop clearance, yaw fallbacks	no durable effect	neutral

Why a better element appeared at all:

1. **The valuable elements are perception diagnoses**: why the object was not found or was wrongly picked. Control tweaks (clearances, yaw fallbacks, planner settings) gave variable results.
2. **A repair's apparent gain can be pure noise.** In the stove repair, iterations 2 and 5 are byte-identical programs (checked with `cmp`) and scored 7/15 and 13/15; the agent itself promoted a caution about this.
3. **Knowledge is not reliably reapplied.** A prompt that worked in two rounds was unused in the third with the library unchanged. The library stored the rule, not the concrete prompt; writers are not required to follow it.

5. Defects and deviations (read before trusting the numbers)

- **Point-prompt API misuse recurred in every later round.** The function takes one `(x, y)` pair; programs passed a list. The allowed-API list in `CLAUDE.md` says `points`, which invites it. I patched round 0 by hand and did not add the warning to the writers' instructions as intended. Trials with the resulting 422 error:

arm	round 1	round 2	round 3
`lex`	120	90	60
`unsolved`	30	15	0

- **Round 0 was patched by hand**: 9 programs (missing `import numpy as np`, list-wrapped point prompt, dict passed as a mask). The log is `outputs/library_*/round_00/patch_log.md`. Later rounds and the noise batches were not patched.

- **Molmo is off**, as in the upstream TC1 runs. It was removed from 11 round-0 programs and the writers were told not to use it.
- **Worker count** differed between the arms (2 against 4); a control showed it is not a source of bias.
- **One run per arm.** Nothing here is a significance statement.

7. Campaign: 30 seeds, 5 repairs (updated as results arrive)

Last update: 2026-10-17, after repetition 2 (both arms).

Design. Same pool, same 3 programs per task, same rules, but every probe uses **30 seeds (51-80)** and each arm makes **5 repairs** (6 probes). The writers' prompts carry the API facts (point prompt takes one `(x, y)` pair, `import numpy as np`, no Molmo), so the point-prompt defect of section 5 is gone. Five repetitions planned; in each, round 0 is written once and shared by both arms. Arm order alternates by repetition. Run ids end in `_s30_r<k>`. Driver: `run_reps.py --reps k --seeds 51..80 --rounds 5 --tag _s30`. Numbers come from `analysis_runs.py`.

Status: reps 1 and 2 done (about 8 to 10 h per arm); rep 3 running; reps 4 and 5 queued.

Noise floor, updated (six independent round-0 generations from the empty library, seeds 51-65): mean 0.067, SD 0.027. The difference of two single runs has SD 0.038. Smallest gap between arms that is reliably seen (5% level, 80% power): 0.106 with 1 repetition per arm, 0.075 with 2, 0.061 with 3, 0.053 with 4, 0.043 with 6. Seed sampling is a small part of this: with 30 seeds it moves a run's mean by only 0.005 to 0.008, so more seeds do not help; more repetitions do.

Mean pass rate over the 27 programs, by round (all seeds of the run).

run	design	0	1	2	3	4	5
`lex` r1 (pilot)	15 seeds, 3 repairs	0.091	0.064	0.057	0.141		
`unsolved` r1 (pilot)	same	0.091	0.067	0.148	0.052		
`lex` r2 (pilot)	same	0.091	0.104	0.126	0.128		
`unsolved` r2 (pilot)	same	0.091	0.111	0.128	0.099		
`lex` s30 r1	30 seeds, 5 repairs	0.028	0.062	0.081	0.048	0.083	0.147
`unsolved` s30 r1	same	0.028	0.063	0.133	0.049	0.094	0.115
`lex` s30 r2	same	0.078	0.152	0.091	0.104	0.072	0.159
`unsolved` s30 r2	same	0.078	0.007	0.070	0.177	0.117	0.100

Final round on seeds 66-80 only (no repair used them): `lex` 0.138 and 0.165, `unsolved` 0.121 and 0.094 (reps 1 and 2).

7.1 `lex` against `unsolved` after two repetitions

rep	`lex` final	`unsolved` final	gap	`lex` gain over round 0	`unsolved` gain over round 0	gap in the mean of rounds 1-5
1	0.147	0.115	+0.032	+0.119	+0.086	-0.007
2	0.159	0.100	+0.059	+0.081	+0.022	+0.021
mean	0.153	0.108	+0.045	+0.100	+0.054	+0.007

- `lex` ends higher in both repetitions, also on seeds 66-80, and gains more over its round 0. Averaged over rounds 1 to 5 the two arms are level, so the final-round gap may be luck at the last step.
- The mean gap of +0.045 is 1.7 standard errors (SE of the mean of two pair differences: 0.038 / sqrt(2) = 0.027). Two repetitions detect a gap of 0.075, so this is not a result. Reps 3 to 5 decide.
- Single rounds are very noisy: `unsolved` in rep 2 fell from 0.078 to 0.007 in round 1 (35% crashed trials) and recovered to 0.177 by round 3.
- All four 30-seed runs end above their round 0 (+0.022 to +0.119); only `lex` rep 1 has a 95% interval above 0, so the gain of the repair loop is likely but is not shown by one run.

7.2 `lex`, repetition 1 (30 seeds, 5 repairs)

- **Result.** The mean rose from 0.028 to 0.147 over the five repairs (+0.119, 95% interval [+0.009, +0.247]; bootstrap over tasks and seeds). On seeds 66-80 it rose from 0.025 to 0.138, so this is not repair-seed overfitting. The final value is about 3 noise SDs above fresh round-0 generations.
- **The path is noisy.** The round steps are +0.034, +0.019, -0.033, +0.035, +0.064; their SD (0.036) is about the noise SD. Only the last step is clearly beyond it. The trend is +0.018 per round.
- **Against the earlier final rounds** (0.141, 0.052, 0.128, 0.099) the final 0.156 is the highest, but only 1.2 noise SDs above their mean, and it had two more repairs and a lower start (0.032 against 0.091). We cannot say 5 repairs beat 3.

- **Crashed trials** are 0 to 19% (4% in the final round) against 13 to 43% in the pilot: the API-facts fix worked.
- **No task is solved on all 30 seeds** in any round. Final-round passes of 90: salad dressing 51, turn on the stove 30, bowl on the stove 19.

round	task repaired	diagnosis	library addition
0	open the middle drawer	handle count unstable, but handle choice did not decide pass or fail	no motion churn without checking known-good seeds; ordinal targets; never retry from a stale pose
1	bowl on the stove	an absolute GraspNet score threshold rejects every grasp; 4 pick attempts help (5 to 11 of 15)	more retries beat tuning; no absolute score filter
2	bowl on the stove	two placement-target clusters from one stove mask	height-bimodality guard; density-mode centroid
3	bowl between plate and ramekin	"ramekin" scores 0.001, so the betweenness test never ran and the wrong bowl was grasped	read the **settled** gripper width, not the return value of `close_gripper()`; no score filter on landmark objects
4	salad dressing	clean trace, "inside footprint", reward 0: release dynamics	one run's pass count cannot validate a change

Only the stove repair of round 1 passed its 15 development seeds with the kept program.

- **One element clearly changed the programs.** The settled-width rule (added after round 3) was used by 0 of 27 programs before it and by **23 of 27 in round 4 and 24 of 27 in round 5**. Programs that use it score higher (0.126 against 0.043), but the 7 programs without it are spread over three tasks, so this shows adoption, not benefit. Best case: salad dressing, 3 passes (round 3) to 42 and 51 (rounds 4 and 5); that task had just been repaired, on the same development seeds.
- **Knowledge crossed tasks.** "Turn on the stove" was never repaired and went 5, 11, 14, 17, 10, 30 of 90, while the stove-bowl and drawer repairs added generic retry and placement rules that appear in its programs.
- **Style against the earlier runs.** The `unsolved` pilots added mostly perception prompts and learned-planner rules; this run added grasp-verification, retry and placement-target rules. The `lex` pilots also stayed with grasp and placement rules, so the style follows the first repaired task more than the selection rule. `lex` picked the bowl on the stove twice (disagreement stayed high after the first, failed, repair).
- **Library size** went from 4.7 KB (L01) to 17.6 KB (L05), additions only.

What the campaign can claim so far. Five repairs on 30 seeds raised this run's pass rate outside the noise, on seeds no repair used. Not shown: that `lex` beats random selection (the paired arm is unfinished), that 5 repairs beat 3, or that any single rule caused the gain.

7.3 Final libraries measured with 6 programs per task

`remeasure.py` writes 3 new programs per task from each final library (read from the run's own library directory) and probes them on the same 30 seeds. With the original 3, every task has 6 programs.

rep	`lex` (6 programs)	`unsolved` (6 programs)	gap	95% interval of the gap
1	0.110 (original 3: 0.147, new 3: 0.074)	0.120 (0.115, 0.125)	-0.009	[-0.151, +0.122]
2	0.104 (0.159, 0.048)	0.101 (0.100, 0.102)	+0.002	[-0.102, +0.090]
mean			-0.003	

- The new programs from the `lex` libraries score far below the original ones (0.074 and 0.048 against 0.147 and 0.159), while the `unsolved` ones match (0.125 and 0.102 against 0.115 and 0.100). The earlier `lex` lead came from the draw of the original programs.
- One process difference: the original programs came from subagents inside the arm's coordinator session, the new ones from a fresh session that read only the library. Both arms are treated alike and `unsolved` shows no shift.
- So with this measure the two arms are level. This is the conclusion to carry forward unless rep 3 and later change it.

8. A second proposition: validate a repair before its lesson enters the library (not tested yet)

Problem. Evaluation is noisy (the SD between independent generations is 0.027; identical programs scored 7/15 and 13/15 on the same seeds), yet a lesson enters the library on the strength of the kept program's dev-seed pass count (rule 3 above). Luck and wrong diagnoses (an early fix loop went 0/15 on dev and 0/50 held out) therefore become permanent library text, and the library only grows.

Hypotheses.

- H4: a library that only accepts repairs that beat their starting program on fresh seeds scores higher on unseen seeds and on unseen tasks than an ungated one at the same repair budget. It can fail: with a margin of 3 passes of 15 the gate accepts about 16% of useless repairs and only about 55% of a true +20-point repair.
- H5: giving the repair agent a trace diagnosis tool (per-query detection scores, grasped-object position, grasp contrast between passing and failing trials) raises the share of repairs that pass the gate.

Pieces built (committed, untested on the simulator): `harness/why.py` (the diagnosis tool, CPU only), `harness/accept.py` (the gate), `prompts/repair_harness_addendum.md`, and `run_reps.py --treatment harness`.

```
promote_with_gate(L, task, start, kept, findings):
    A <- pass count of start on acceptance seeds 81..95          # seeds used by no repair and no probe
    B <- pass count of kept on the same seeds
    if B - A >= 3: L <- add lessons from findings to L
    else:          L unchanged; log "rejected (A, B)"
```

Planned design: arms `unsolved`, `unsolved + tool`, `unsolved + gate`, `unsolved + both`, at least 4 repetitions, budget counted including the gate's trials, measured on unseen seeds, 6 programs per task, and on the 21 held-out tasks. Section 6 lists the held-out transfer test as the first thing to run after the campaign.

Where things are, by experiment id

All paths are under `aspire/sim/outputs/` (gitignored). The live directory `.claude/libero/skills/` is only a working copy: each run saves its own libraries, so it is restored to the committed version when no run is active.

experiment id	what it is	libraries	probes and selections	repairs
`library_lex_r1`	arm `lex`, 3 repairs + final probe	`library/L00` (empty) to `L03`	`round_NN/probe.json`, `round_NN/selection.json`	`round_NN/fixloop/<suite>/<task>/findings.md`, `skill_promotions.jsonl`
`library_unsolved_r1`	arm `unsolved` (also holds the shared round 0)	`library/L00` to `L03`	same	same
`library_noise_a`, `library_noise_b`	fresh round-0 generations from the empty library	`library/L00`	`round_00/probe.json`	none
`library_lex_r2`, `library_unsolved_r2`	pilot repetition 2 (15 seeds, 3 repairs; shared round 0)	`library/L00` to `L03`	same	same
`library_lex_s30_r<k>`, `library_unsolved_s30_r<k>`	campaign (k = 1 to 5); `outputs/remeasure/<run>` holds the 3 extra programs per task and their probe (30 seeds 51-80, 5 repairs, 6 probes; shared round 0 per repetition)	`library/L00` to `L05`	same	same
`skill_snapshots/family_start`	skills after the first single-task fix loop; equals the committed `.claude/libero/skills`	5 files	none	none

Each run also has `state.json` (pool, rounds, settings) and `coordinator.md` (the exact prompt it was started with). Round 0 of `library_lex_r1` is a byte-identical copy of the shared one (`probe.json` compared with `cmp`).

6. What this supports and what comes next

Supported: the pipeline runs end to end; a perception-level skill can raise a task's pass rate in later rounds; library edits can also lower it; run-to-run noise is as large as the differences seen.

Not supported: that disagreement selection beats random selection, in either direction.

Next, in order:

1 ~~Add the point-prompt line and `import numpy as np` to the writers' instructions~~ (done for the campaign).

2. ~~Repeat each arm; more seeds and rounds~~ (section 7, running).
3. Record the concrete perception prompt that worked in the library, not only the rule.
4. After the campaign: evaluate the start library and each arm's final library on held-out tasks (21 swap tasks lie outside the pool).